

Plano: Consolidação Histórico Planilha → ClickUp + Rollout do App Timesheet

Context

Hoje o controle de alocação de horas do Grupo SAL vive em **00_GRUPO-SAL_CONSOLIDADO_ALOCACAO_2026** (Google Sheets), alimentado por 4 planilhas-fonte por unidade de negócio (Branding / Comunicação / Conteúdo / CSC). O webapp Timesheet já está pronto e escreve em ClickUp (**LISTA_REGISTRO_ALOCACAO** em BACK_OFFICE > [ALOCAÇÕES]), mas só será adotado se **toda a visão de 2026** estiver no ClickUp — caso contrário, gestores teriam que olhar dois lugares.

Objetivos:

1. Migrar o histórico de 2026 da planilha para o ClickUp num formato compatível com o app.
2. Definir o plano de adoção (rollout) para gestores e colaboradores começarem a lançar pelo app a partir de uma semana definida.

Estado de partida (verificado via CSV export em 2026-05-13):

- Planilha tem **1.851 linhas**, **W01-W20/2026** (W20 = semana corrente, 11-15/05; alguns lançamentos sem **horas_realizadas** ainda). **Não há gap** — o consolidado está em dia.
- **LISTA_REGISTRO_ALOCACAO** tem **~151 tasks no formato antigo** "**Pessoa → Projeto (Sem N)**", incompatíveis com o **makeTaskName()** do app (**src/lib/clickup/entries.js:33**) — o app não deduplica essas e criaria duplicatas.
- **LISTA_PESSOAS** está **incompleta** (~5 pessoas cadastradas; planilha tem 37).
- **LISTA_PROJETOS** está razoavelmente populada, mas alguns dos 60 projetos da planilha podem não ter equivalente exato.
- **CSV congelado em** **docs/00_GRUPO-SAL_CONSOLIDADO_ALOCACAO_2026 - REGISTRO_ALOCACAO_CONSOLIDADO.csv** (174 KB) — fonte de verdade para o import.

Decisões já tomadas pelo usuário:

- Apagar as 151 tasks antigas e reimportar tudo do zero a partir da planilha.
- **Descartar a coluna CARGO** — não cabe no MVP (pode virar custom field depois se virar requisito de relatório).
- Escopo deste plano: **script de import 1x + plano de rollout**. Não inclui mudanças no app, dashboard ou governança operacional (ficam para iterações futuras).

Fase A — Migração de dados (planilha → ClickUp)

A.0 — Tornar pessoas/projetos auto-gerenciáveis (pré-requisito permanente)

Problema permanente: hoje `createEntry()` em `src/lib/clickup/entries.js:107` busca person/project no cache de `LISTA_PESSOAS` / `LISTA_PROJETOS` e, se não encontra, **cria a entry mesmo assim mas deixa os relationship fields vazios silenciosamente** (linhas 129–138). Isso significa:

- Hoje: 32 pessoas / 17+ projetos da planilha não estão nas Listas → import vai criar entries sem `rel_colaborador` / `rel_projeto`
- Em 2027: qualquer projeto novo do ano vai falhar do mesmo jeito, sem aviso. Falha silenciosa que vai acumular.

Solução permanente — auto-criar on demand:

1. Adicionar em `src/lib/clickup/entries.js` duas funções públicas:

```
```js
```

```
export async function ensurePerson(name) { / check cache → create in LIST_PEOPLE → cache → return id / }
```

```
export async function ensureProject(name) { / idem para LIST_PROJECTS / }
```

```
```
```

1. Reescrever o bloco de relationship em `createEntry()` (linhas 129–138) para usar `ensurePerson` / `ensureProject` em vez de só `personIdCache.get()`. Erros de criação **não devem ser silenciados** — devem propagar para a UI mostrar "Erro ao registrar [nome] em LISTA_PESSOAS, tente de novo".
2. Mesma garantia em `upsertForecast()` e em qualquer caminho que crie entry — sempre que um nome aparece, ele existe nas Listas ao final da operação.

Por que dentro do app, e não só no script:

- O app (aba **Planejamento**) é o canal principal pelo qual novos projetos entram em 2026/2027/+. Se a auto-criação só vive no script de import, na virada do ano qualquer projeto novo precisa de cadastro manual em duas Lists antes de poder ser usado — cria fricção e quebra a promessa "o app é a fonte de verdade".
- O script de import e o app **chamam exatamente a mesma função**. Isso elimina a chance de divergência: o que o app faz na sexta-feira é o mesmo que o batch faz na migração.

Trade-off aceito: typo na UI cria registro permanente. Mitigações no app (já existentes ou a planejar separadamente, **fora do escopo deste plano**): combobox com autocomplete dos nomes existentes; quando o usuário digita algo que tem >85% similaridade com um nome existente, sugerir; botão "renomear" no `Directory.jsx` para corrigir typo retroativamente.

Arquivos a modificar:

- `src/lib/clickup/entries.js` — adicionar `ensurePerson` , `ensureProject` ; refatorar `createEntry` .
- `src/lib/clickup/lists.js` — já tem CRUD de people/projects; reutilizar `createPerson` / `createProject` se existirem.
- Sem mudanças na UI nesta fase (próxima fase pega o autocomplete).

Verificação: após o fix, criar manualmente uma entry no app com um nome de projeto novo ("Projeto Teste 2027") e confirmar que ele aparece em LISTA_PROJETOS no ClickUp imediatamente.

A.1 — Pré-trabalho de limpeza

✅ Snapshot CSV já existe em docs/00_GRUPO-SAL_CONSOLIDADO_ALOCACAO_2026 - REGISTRO_ALOCACAO_CONSOLIDADO.csv (1.851 linhas, W01-W20). Antes de rodar o import, mover/copiar para data/historico-2026-snapshot.csv e commitar (assim o snapshot fica congelado e auditável).

Validação automática das 37 pessoas e 60 projetos da planilha contra LISTA_PESSOAS / LISTA_PROJETOS é feita pelo script (passo A.3.2).

A.2 — Apagar as 151 tasks legadas

Criar scripts/wipe-registro-alocacao.mjs — pequeno, isolado, com --dry-run por padrão e --confirm para executar de fato.

- Pagina /list/{LIST_ENTRIES}/task?include_closed=true&page=N até esgotar.
- Filtra tasks cujo nome **NÃO** segue o regex ^\d{4}-W\d{2} \| \. + \| \. +\$ (mantém a 1 task do Juarez no formato novo, se quiser; ou apaga tudo — preferível).
- DELETE /task/{id} com 250 ms entre chamadas (folga sobre 100 req/min do plano Free).
- Loga cada exclusão. Reusa o boilerplate de scripts/populate-relationships.mjs (auth, retry 429).

A.3 — Script de import principal

Arquivo: scripts/import-historico-2026.mjs

Entrada: data/historico-2026-snapshot.csv (header real: DATA_START, DATA_END, SEMANA, MÊS, PROJETO, UNIDADE_DE_NEGOCIO, PESSOA, CARGO, TIPO, Nº_HORAS_PREVISTO, Nº_HORAS_REALIZADO — note o espaço extra em DATA_START / DATA_END).

Lógica (em ordem):

1. Parse + normalização:
 2. Year = ano de DATA_START (cuidado: W01 começa em 29/12/2025 — usar o ano de DATA_END ou simplesmente forçar 2026 para o consolidado, já que a planilha só cobre o ciclo 2026).
 3. Hours_Forecast = Nº_HORAS_PREVISTO ; Hours_Consolidated = Nº_HORAS_REALIZADO (ambos integers).
 4. Ignorar CARGO e TIPO .
1. Auto-criação de pessoas/projetos faltantes (lógica compartilhada com o app — ver Fase A.0):
 2. O script chama a mesma função ensurePerson(name) / ensureProject(name) que o app usa em createEntry . Auto-cria no ClickUp se não existir, com cache de sessão.
 3. Diferença do script vs. app: o script roda em modo --strict-confirm — antes de auto-criar, faz dry-run que imprime todas as pessoas/projetos novos que seriam criados, e exige --

`confirm` para prosseguir. Isso evita typo na planilha virar registro permanente.

4. Sem flag, o comportamento default é o do app: auto-criar silenciosamente. Para uso humano interativo do app isso é o certo; para batch import é arriscado, daí o gate.

1. Criação das entries (1.851 chamadas):

2. Para cada linha do CSV, montar payload **idêntico ao `createEntry()` de `src/lib/clickup/entries.js:107`**, mas setando **tanto `horas_previstas` quanto `horas_realizadas`** (na planilha, em geral previsto == realizado; quando `horas_realizadas` está vazia — semana corrente — passar `null`).
3. `name = makeTaskName(Year, Week, Person, Project)` — replicar a função.
4. `centro_de_custo` → `ccNameToId()` (já existe a tabela hardcoded em `fields.js`, replicar no script). **Atenção:** CSC apareceu na planilha (2 linhas) e já está no dropdown — confirmar mapeamento.
5. Após cada criação, popular `rel_colaborador` e `rel_projeto` via `POST /task/{id}/field/{fieldId}` no formato `{ value: { add: [linkedTaskId] } }` (ver memo `feedback_clickup_relationship_fields`).
6. Throttle: **200 ms entre tasks + 150 ms entre os 2 relationship updates** → ~600 ms por linha → **~18 min para 1.851 linhas**.
7. Tratamento de 429 já vem do helper.

1. Idempotência:

2. Antes de criar cada task, manter `Set<taskName>` carregado no início (de fato — depois do wipe, deveria estar vazio).
3. Se `taskName` já existir, **pular** (não atualizar). Permite re-rodar o script com segurança.

1. Logs e saída:

2. Imprimir contador `[N/1851]` por linha.
3. Ao final: total criadas, total puladas, total erros, e gravar `data/import-log-{timestamp}.json` com cada (taskName → taskId | erro).

A.4 — Verificação pós-import

- `clickup_filter_tasks` na list, `include_closed=true`, confirmar `count == 1851` (assumindo nenhum erro).
- Spot-check de 3 linhas (W01, W10, W20) abrindo a task no ClickUp e conferindo todos os 9 campos contra a planilha.
- Abrir o app, ir para uma semana já importada, conferir que a UI mostra os dados corretamente.

Arquivos críticos a referenciar:

- `src/lib/clickup/entries.js:33,107` (makeTaskName e createEntry — replicar shape exatamente)
- `src/lib/clickup/fields.js:7,21` (FIELDS e CENTRO_DE_CUSTO_OPTIONS — copiar para o script)
- `scripts/populate-relationships.mjs` (boilerplate de auth/retry)

Fase B — Rollout para a empresa

A migração cobre **W01–W20 (todo o ano de 2026 até hoje)**. Não há gap — o rollout pode começar na próxima semana sem reconstrução retroativa.

B.1 — Semana de corte

Proposta: definir **W21 (18–22/maio/2026)** como semana de corte — a partir dela, o controle é feito **exclusivamente no app**. Toda a base histórica de 2026 (W01–W20) já estará no ClickUp via Fase A.

A W20 (corrente) fica como semana de transição: foi lançada na planilha, vai estar no ClickUp via import, e o app já pode ser usado para validar/ajustar as **horas_realizadas** dela na sexta.

B.2 — Anúncio interno (1 semana antes do corte)

E-mail/Slack/Notion da diretoria, contendo:

1. **Por quê:** o controle de alocação agora é no app, pra dar visibilidade direta dos gestores aos dados sem depender de planilha consolidada.
2. **O que muda:**
3. **Gestores** (já existentes na planilha): aba **Planejamento** do app → na **segunda-feira** lançam previsão de horas das suas equipes para a semana.
4. **Colaboradores:** aba **Timesheet** → na **sexta-feira** confirmam horas realizadas (ou ajustam se variou da previsão).
5. **A partir de quando:** semana de corte (ex: 18/05).
6. **Onde acessar:** <https://gruposal.github.io/timesheet>.
7. **Como configurar:** colar token ClickUp pessoal nas Configurações (⚙️) do app — link para tutorial curto.

B.3 — Material de apoio (criar antes do anúncio)

- **GIF/vídeo curto (~2 min)** mostrando: login (config do token) → aba Planejamento (gestor lançando) → aba Timesheet (colaborador confirmando).
- **Página de FAQ no Notion / Google Doc:**
 - "Como gero meu token ClickUp"
 - "Esqueci de lançar a semana passada — o que fazer?"
 - "Meu projeto não aparece na lista — como cadastrar?"
 - "Cap de 40h — o que acontece se eu exceder?"

B.4 — Treinamento ao vivo (~30 min)

Uma única sessão pelo Google Meet, gravada. Apresenta o app, mostra fluxo gestor + colaborador, abre para Q&A. Convocar:

- Todos os gestores (preferência: presença obrigatória)

- 1–2 representantes por unidade (vão virar referência local)
- Demais colaboradores: opcional (assistem a gravação)

B.5 — Acompanhamento das 2 primeiras semanas

- Toda **sexta às 17h**, verificar via dashboard ou query MCP:
- Quantas pessoas têm <40h lançadas naquela semana?
- Quantos gestores não preencheram a previsão da semana seguinte?
- Cobrar diretamente os faltantes pelos canais habituais.

B.6 — Critério de sucesso (definir antes do go-live)

- **Semana 1 pós-corte:** ≥80% das pessoas com 40h lançadas até segunda às 10h.
- **Semana 4:** ≥95%, e planilha consolidada Google **desativada** (read-only / arquivada).

Verificação end-to-end

1. Após Fase A:

```
""bash
# Conferir contagem
# via MCP: clickup_filter_tasks list_ids=["901326824645"] include_closed=true
# esperado: 875 tasks (ou 876 se mantiver a do Juarez)
""
```

Spot-check abrindo 3 tasks aleatórias no ClickUp: todos os 9 custom fields preenchidos + 2 relationships ligadas.

1. No app (**npm run dev**):

2. Configurar token, navegar para **2026-W05** → conferir que pessoas/projetos da semana aparecem com horas certas.
3. Editar uma entrada, salvar, conferir no ClickUp que atualizou em vez de duplicar.

1. Após Fase B (rollout):

2. Dashboard semanal (Domingo): comparar previsão vs. realizada para a semana corrente.
3. Verificar critério de sucesso (B.6) por 4 semanas seguidas.

Riscos e mitigações

| Risco | Mitigação |
|--|--|
| Planilha atualizada durante a migração | Trabalhar com snapshot CSV congelado (A.1). Reimport pode ser repetido se necessário (script é idempotente). |
| Projetos com nomes ligeiramente diferentes entre planilha e LISTA_PROJETOS | Script pausa em A.3.2 e exige confirmação explícita antes de criar novos. Permite normalização manual prévia se preferir. |
| Plano Free do ClickUp (100 req/min) bloqueia o import | 200 ms entre tasks + retry automático em 429. ~18 min total — bem dentro do limite. |
| Usuários não adotam o app pós-rollout | Critério de sucesso (B.6) força acompanhamento. Treinamento ao vivo + cobrança direta nas 2 primeiras semanas. |
| MCP do Google Drive lê versão cacheada (caso já visto) | CSV manual exportado é a fonte de verdade. Para qualquer auditoria pós-import, comparar contra o CSV congelado, não contra o MCP read. |